Head in the Clouds: A Cloud-Native Trek Booking Platform on Azure

Student Details

- Student Name(s):
  - Navon John Lukose, 240905602 (Roll 61)
  - Srishti Sudesh, 240905626 (Roll 62)
- Semester: 5th Semester
- Section: C
- Batch: *2*
- Team:
- Academic Year: 2026-27
- Department: School of Computer Engineering

Supervisor Details

- Supervisor Name: Mohandas Shenoy P
- Department: School of Computer Engineering

Synopsis / Abstract

Head in the Clouds is a cloud-native web application for running guided trekking trips, deployed on Microsoft Azure and orchestrated by Kubernetes. The system replaces the spreadsheets and phone calls that small trekking operators use today with one shared set of records and three roles over it. An administrator publishes routes and manages people, trek staff run the treks assigned to them, and trekkers search, book and track their trips. The application layer is a stateless Flask JSON API and a Vue single-page front end, and all of the state sits in managed services. Azure Database for PostgreSQL holds the relational data, Azure Managed Redis serves as the response cache, and background work is queued on Azure Service Bus and triggered on a timer by Azure Functions. Every workload runs as a container on Azure Kubernetes Service, provisioned by Terraform, packaged by Helm and released by a GitHub Actions pipeline. The project is as much about the operational side as the application. It covers autoscaling under a seasonal load, a booking path that stays correct across several replicas, zero-downtime rolling releases, and infrastructure that can be destroyed and rebuilt from code.

Problem Statement

Small trekking operators coordinate their trips over spreadsheets, WhatsApp groups and phone calls. Trek approvals, registrations, slot availability and payment records live in different places and disagree with each other, so seats get double-booked, cancellations are missed, and nobody has a reliable list of who is actually on a trek that leaves tomorrow.

The operational side is what makes this a cloud problem as well as a software one. Demand is sharply seasonal. An operator sees almost no traffic for weeks, then a

burst of concurrent bookings in the minutes after a popular route opens. A single hand-configured server is either idle and paid for or too small at exactly the moment it matters, and it fails as one unit, taking the booking system down during the only window that generates revenue. A correct booking rule is also harder than it looks once more than one server is involved, because two requests can read the same last free slot and both conclude it is available.

These failures have real consequences. An oversold trek means someone is turned away at a trailhead, and a lost booking record means an operator cannot account for who is in the mountains.

Proposed Solution

The proposed solution is a cloud-native trekking management platform on Azure, built as a stateless application tier over managed data services and scheduled by Kubernetes. It consists of six main modules.

Authentication and Role Module:
Registration and token-based login for all three roles from a single user table, with roles held in a many-to-many join and enforced by a decorator on every view. Authentication uses a bearer token instead of a server-side session, which is what allows any API pod to serve any request.

Trek Catalogue Module:
Administrators create and approve routes with difficulty, duration, location, fees and slot counts, and assigned staff set status and slots on their own treks only. The public listing is filterable and cached in Azure Managed Redis, so a burst of browsing traffic is absorbed before it reaches the database.

Booking Module:
Search, book, cancel and history for trekkers. Slot allocation is a single conditional UPDATE in PostgreSQL that decrements the count only while a slot is free and reports how many rows it changed, so the last seat cannot be sold twice no matter how many API replicas are running. This module is the reason for choosing a real transactional database over anything eventually consistent.

Asynchronous Jobs Module:
Celery workers run as an autoscaled deployment consuming an Azure Service Bus queue. The recurring schedule is held by timer-triggered Azure Functions instead of a Celery beat pod, so no singleton process has to stay alive. The jobs send trek reminders three days out, compile a monthly activity report for the administrator through Azure Communication Services Email, and generate CSV booking exports on demand. Each export is written to Azure Blob Storage and handed back as an expiring SAS link, never as a file on a pod. Service Bus delivers at least once, so every task is written to be safe to run twice, and messages that keep failing move to the queue’s dead-letter subqueue.

Cloud Infrastructure Module:
The whole environment is defined as code. Terraform provisions the virtual network, the AKS cluster, the zone-redundant PostgreSQL server, Azure Managed Redis, the Service Bus namespace, the Functions app, the storage accounts, Azure Front Door with its WAF policy, the Azure DNS zone, Azure Container Registry, Key Vault, the

managed identities with their workload identity federation, and the monitoring resources. Helm packages the cluster workloads with one values file per environment. Nothing is clicked in the portal, and the entire stack can be destroyed and recreated.

Delivery and Observability Module:
A GitHub Actions pipeline tests the code, builds container images tagged by commit SHA, pushes them to Azure Container Registry and runs a Helm upgrade against staging. Production sits behind a manual approval and can be rolled back with one command. Horizontal Pod Autoscalers scale the API on CPU, KEDA scales the workers on queue length, and node auto-provisioning adjusts node capacity underneath. Azure Monitor Container Insights collects logs and metrics, with alerts on 5xx rate, queue length, dead-lettered messages and database storage.

Each module answers one of the three failures in the problem statement. The records disagree, so there is one database of record. The load is spiky, so the tier that handles it is stateless and autoscaled. The booking rule breaks under concurrency, so correctness is pushed down to a single atomic statement in the database.

Tools and Technologies

- Cloud Service Provider: Microsoft Azure, single subscription, Central India region across three availability zones
- Cloud Managed Services you plan to use: Azure Kubernetes Service with node auto-provisioning and the KEDA add-on; Application Gateway for Containers as the cluster ingress, driven by the Gateway API; Azure Front Door Premium with a WAF policy carrying rate-limit rules on the login and booking endpoints; Azure DNS; Azure Blob Storage for the static front end and for generated exports; Azure Service Bus as the job queue, with its dead-letter subqueue; Azure Functions on a timer trigger for the recurring jobs; Azure Container Registry for images; Azure Communication Services Email for outbound mail; Azure Key Vault through the AKS Secrets Store CSI driver; Microsoft Entra Workload ID so no credential is stored in the cluster; Azure Monitor with Container Insights, a Log Analytics workspace and action groups for alerts
- Databases you plan to use: Azure Database for PostgreSQL flexible server, PostgreSQL 17, with zone-redundant high availability, automated backups and point-in-time restore, as the system of record; Azure Managed Redis as the response cache and the Celery result backend
- Source version control: Git, hosted on GitHub, short-lived branches merged to main by pull request, with GitHub Actions as the CI/CD pipeline and images tagged by commit SHA
- Tools you plan to use: Terraform with the azurerm provider for infrastructure, with remote state in Azure Blob Storage locked by blob lease; Helm 4 for cluster workloads; Docker, kubectl and the Azure CLI; Alembic for schema migrations run as a pre-upgrade Job; Flask, Flask-SQLAlchemy, Flask-Security-Too and Celery on the backend; Vue 3, Vue Router, Vite, Bootstrap 5 and Chart.js on the front end; pytest for tests and graphviz for the generated ER diagram

Expected Outcomes

The expected outcome is a working trekking management platform running on Azure, reachable over HTTPS, with its entire infrastructure reproducible from Terraform and Helm in a fresh subscription. Success is measured on both halves of the project.

On the application side, the system is successful if all three roles can complete their work end to end. An administrator publishes a trek and assigns staff, staff open it and see their participant list, and a trekker books, cancels and exports a history, with the reminder and monthly report jobs firing on schedule.

On the cloud and DevOps side, the targets are concrete. A load test firing concurrent bookings at the last free slot on a trek must oversell zero seats. A release must go out with no dropped requests, verified by rolling a new image while traffic is running. The API must scale out under load and back in when it subsides without manual intervention. Killing an API pod, and separately forcing a failover of the PostgreSQL server, must leave the service usable. Finally, `terraform destroy` followed by `terraform apply` must rebuild the environment from nothing, which is the real test of whether the infrastructure is code or a description of something built by hand.

Tentative Timeline

Week #	Phase/Tasks
1-4	Requirement analysis and specification; Azure subscription, budget alerts and baseline virtual network
4-5	High level design: service selection, network layout, cost estimate
5-6	Low level design (UI/UX, backend API, database schema, Terraform module layout, Helm chart structure)
4-7	Implementation, containerisation, unit testing and debugging
7-9	Testing: system and integration testing, concurrency and load testing of the booking path, failover drills
9-10	Deployment and release: CI/CD pipeline, staging then production, autoscaling and alerts tuned
11	Documentation (design documentation, test results, presentation ppt) preparation
12	Final presentation